v1.2.1
20260401

# Programmazione Concorrente e Distribuita

Ingegneria e Scienze Informatiche - UNIBO

a.a 2025/2026

Prof. Alessandro Ricci

# [module-2.2]
# REACTIVE PROGRAMMING

# SUMMARY

- Reactive programming (RP) overview
  - research roots
    - reference paper:
      - Engineer Bainomugisha, Andoni Lombide Carreton, Tom van Cutsem, Stijn Mostinckx, and Wolfgang de Meuter. 2013. *A survey on reactive programming*. ACM Comput. Surv. 45, 4, Article 52 (August 2013)
  - basic abstractions
    - behaviours and events
- RP in the mainstream: Reactive extensions (Rx)
  - Observable, observers

# REACTIVE PROGRAMMING

- Both the CPS and Promises assume that asynchronous computations will deliver their result in *one shot*
  - however it is not uncommon in application the need to manage (asynchronous) **streams** of data/events.
  - this is the goal of **Reactive Programming**
    - *asynchronous data stream programming*
- Reactive Programming paradigm
  - oriented around ***data flows*** and the propagation of change
  - easing the management of asynchronous streams of data and events
    - strongly related to the Observer pattern & event-driven programming
  - originally introduced in 1990s, it is attracting much attention today as an effective approach for developing responsive (web) apps, in particular in the context of Big data

# RP LANGUAGES AND FRAMEWORKS

- *Reactive extensions* have been introduced for all major programming languages/systems
  - FrTime (based on Scheme)
  - Microsoft's Reactive Extensions (Rx)
  - Reactive JavaScript - Flapjax, Bacon.js, AngularJs
  - Reactive DART
  - Scala.React
  - RxJava
  - …

# REACTIVE PROGRAMMING ROOTS

- Research roots
  - most of the research on reactive programming descends from the language Fran [Elliott and Hudak 1997]
    - a functional domain specific language developed in the late 1990s to ease the construction of graphics and interactive media applications
    - **Functional Reactive Programming (FRP)**
  - based on the *synchronous dataflow programming* paradigm [Lee and Messerschmitt 1987] but with relaxed real-time constraints.
- Recently proposed as a solution to deal with the problems and complexity related to the development of event-driven applications

# ASYNCHRONOUS PROGRAMMING PROBLEMS

- Asynchronous programs and applications
  - difficult to program using conventional sequential programming approaches, because *it is impossible to predict or control the order of arrival of external events*
    - control jumps around event handlers as the outside environment changes unexpectedly
- The ***Inversion of control*** problem
  - the control flow of a program is driven by external events and not by an order specified by the programmer
  - when there is a state change in one computation or data, the programmer *is required to manually update all the others that depend on it*
  - such manual management of state changes and data dependencies is complex and error-prone (e.g., performing state changes at a wrong time or in a wrong order) [Cooper and Krishnamurthi 2006]

# ASYNCHRONOUS PROGRAMMING PROBLEMS

- Coordinating callbacks can be a very daunting task even for advanced programmers
  - numerous isolated code fragments can be manipulating the same data and their order of execution is unpredictable
  - “asynchronous spaghetti”
- **Since callbacks usually do not have a return value, they must perform *side effects* in order to affect the application state** [Cooper 2008].

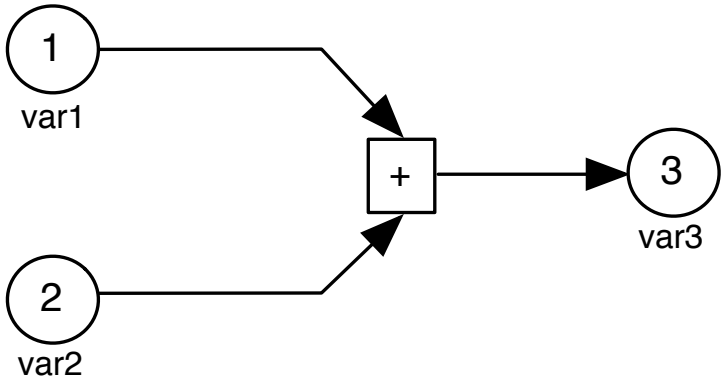
# REACTIVE PROGRAMMING PARADIGM

- RP paradigm
  - providing abstractions to express programs as *reactions* to external events…
  - … but *having the language automatically manage the flow of time (by conceptually supporting simultaneity), and data and computation dependencies.*
    - abstracting over time management, just as garbage collectors abstract over memory management
- RP facilitates the ***declarative development* of event-driven** applications
  - developers can express programs in terms of what to do, and let the language automatically manage when to do it
  - built around the notion of ***continuous time-varying values and propagation of change***
    - state changes are automatically and efficiently propagated across the network of dependent computations by the underlying execution model.

# CHANGE PROPAGATION EXAMPLE

- Consider a simple example of calculating the sum of two variables.

```
var1 = 1
var2 = 2
var3 = var1 + var2
```




- In reactive programming, the value of the variable var3 is always kept up-to-date.
  - the value of var3 is automatically recomputed over time whenever the value of var1 or var2 changes
- Values change over time and when they change all dependent computations are automatically re-executed
- In reactive programming terminology, the variable var3 is said to be *dependent* on the variables var1 and var2

# RP ABSTRACTIONS

- Reactive programming abstracts *time-varying events* for their consumption by a programmer
  - the programmer can then define elements that will react upon each of these incoming time-varying events
  - abstractions are first-class values, and can be passed around or even composed within the program.
- Two main kinds of *reactive abstractions*:
  - **event streams**
    - modeling discrete (or sparse) time-varying values
    - asynchronous abstractions for the progressive flow of intermittent, sequential data coming from a recurring event
    - e.g. mouse events can be represented as event streams.
  - **behaviours (or signals)**
    - continuous values over time
    - abstractions that represent uninterrupted, fluid flow of incoming data from a steady event.
    - e.g. a timer can be represented as a behaviour.

# EXAMPLE IN FLAPJAX

- A timer in Flapjax

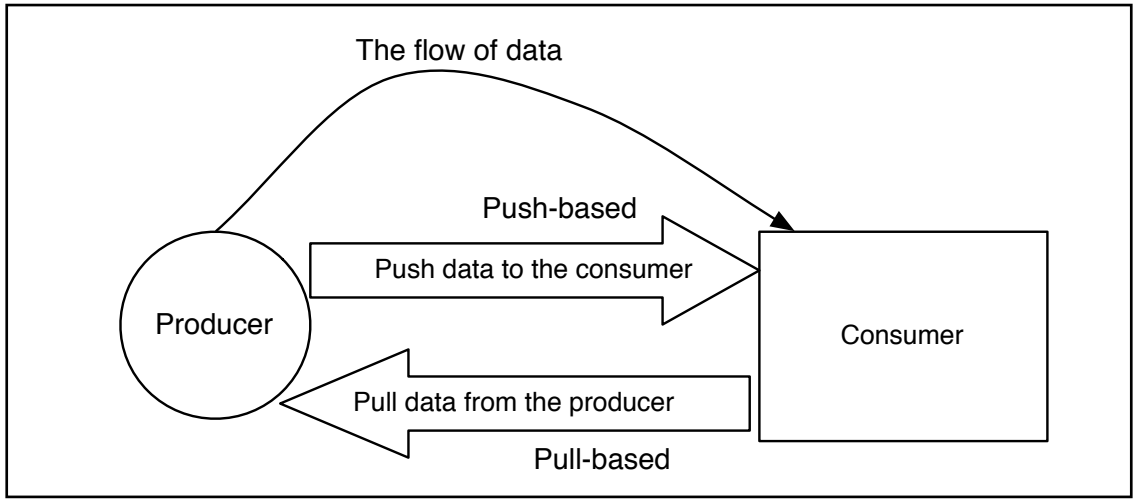
```
var timer = timerB(100);
var seconds = liftB(
  function (time){
    return Math.floor(time / 1000);
  }, timer );
insertDomB(seconds, 'timer-div');
```

- This examples set up a DOM element in web page showing a timer count in seconds.
  - first a timer behaviour producing a value after every 100 milliseconds is created (timer)
    - timer is a behaviour variable
  - to show the time in second, a further behaviour variable is defined (seconds), storing the flow in seconds
  - finally the "seconds" behaviour is inserted within the DOM element with the ID timer-div on a webpage
    - the value of the timer will therefore be continuously updated and displayed on the page
- Key point:
  - fully declarative way of managing events and event streams

# EVALUATION MODEL

- The evaluation model of a reactive programming language is concerned with *how changes are propagated across a dependency graph of values and computations.*
- From the programmer's point of view, propagation of changes happens automatically
  - a change of a value should be automatically propagated to all dependent computations
  - when there is an event occurrence at an event source, dependent computations need to be notified about the changes, possibly triggering a recomputation
- At the language level, the design decision that needs to be taken into account is who initiates the propagation of changes
  - whether the source should "push" new data to its dependents *(consumers)*
  - or the dependents should "pull" data from the event source *(producer)*

# PULL AND PUSH MODELS

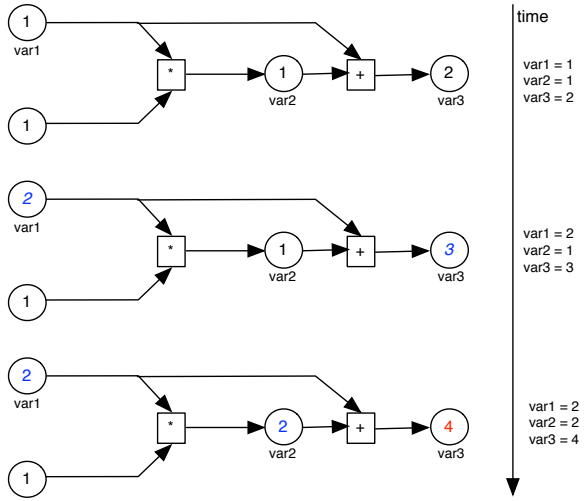



- Two evaluation models
  - **Pull-based -** the computation that requires a value needs to "pull" it from the source.
    - propagation is driven by the demand of new data (demand-driven)
    - can be implemented by lazy languages (e.g. Haskell)
  - **Push-based -** when the source has new data, it pushes the data to its dependent computations.
    - propagation is driven by availability of new data (data-driven) rather than the demand.
    - implemented by eager languages (e.g. Flapjax based on JavaScript, Scala.React)

# GLITCHES

- Glitches are update inconsistencies that may occur during the propagation of changes
  - when a computation is run before all its dependent expressions are evaluated, it may result in fresh values being combined with stale values, leading to a glitch [Cooper and Krishnamurthi 2006].
  - problem affecting only the push-based evaluation model
- Example

```
var1 = 1
var2 = var1 * 1
var3 = var1 + var2
```




The value of the variable var2 is expected to always be the same as that of var1, and that of var3 to always be twice that of var1. When the value of var1 is 1, the value of var2 is 1 and var3 is 2.

If the value of var1 changes to, say 2, the value of var2 is expected to change to 2 while the value of var3 is expected to be 4. However, in a naive reactive implementation, changing the value of var1 to 2 may cause the expression var1 + var2 to be recomputed before the expression var1 * 1. Thus the value of var3 will momentarily be 3, which is incorrect. Eventually, the expression var1 * 1 will be recomputed to give a new value to var2 and therefore the value of var3 will be recomputed again to reflect the correct value 4.

# GLITCH AVOIDANCE

- Most recent reactive implementations achieve glitch avoidance in reactive programs running on a single computer, but not in distributed reactive programs
- Avoiding glitches in a distributing setting is not straightforward because of network failures, delays and lack of a global clock
  - open research issue [Salvaneschi et al 2014, Mogk et al. 2018]

# LIFTING

- The values in expressions that depend on the reactive values *need to be reactive themselves.*
- A variable which gets assigned with some expression involving behaviours or event streams becomes a reactive variable itself
  - since updates from the behaviours/event streams will affect the variable itself
- The process of converting a normal variable into a reactive variable is called **lifting**
- In order to correctly recompute all the reactive expressions once an event stream or behaviour is triggered, most libraries construct a *dependency graph* behind the scenes.
  - whenever an expression changes, the dependent expressions are recalculated and their values updated
  - In the timer example, a time change that happens updating timer triggers the re-evaluation of the function (time){...} and the update of seconds, which subsequently updates the value inserted in the web page

# IMPLICIT VS. EXPLICIT LIFTING

- Implicit vs. explicit lifting
  - a number of JavaScript libraries implicitly perform lifting for the programmer – e.g. in Bacon.js.
  - for some, the programmer has to perform lifting of reactive expressions manually – such as in React.js
  - a third category of libraries offer both implicit and explicit lifting – for instance Flapjax
  - which if used as a library the programmer perform lifting explicitly but if used as a compiler the code is transformed implicitly

# COMPOSING EVENT STREAMS AND BEHAVIOURS

- A essential feature in reactive programming is ***reactive abstraction composition***
  - this allows to avoid the callback nightmare described previously
  - for instance, instead of having three separate callbacks to separately handle mouse clicks, mouse moves or mouse releases, we can compose them as a single event stream which responds to all the three events.
- Most JavaScript libraries provide this kind of fundamental support.
  - for example, in Bacon.js, properties (or behaviours) can be composed using the combine operator
  - in Flapjax, mergeE function is provided, which throttles the latest stream that comes in from either event.

# COMPOSING EVENT STREAMS

- Example in Flapjax:

```
var saveTimer = timerE(10000); //10 seconds
var saveClicked = extractEventE('save-button','click');
var save = mergeE(saveTimer,saveClicked);
save.mapE(doSave); //save received data
```

- In this example three event streams are created:
  - saveTimer is an event stream is created, that is triggered after every 10 seconds
  - saveClicked is an event stream that is triggered whenever a user clicks on the save-button
  - save is triggered whenever either the timer elapses or the save button is clicked
  - finally, for each element of the save stream, the doSave function is called (functional style)

# ONGOING RESEARCH DIRECTIONS

- Among the advanced aspects currently explored in the research:
  - distributed reactive programming
    - composing reactive distributed streams
  - handling fault tolerance

# AVAILABLE REACTIVE LIBRARIES

- Integrating reactive programming in existing programming languages/paradigms
- Main examples
  - **Reactive Extensions (Rx)**
    - an API for asynchronous programming with observable streams, available on multiple platforms
    - http://reactivex.io/
  - **Reactive Streams**
    - Initiative to provide a standard for asynchronous stream processing with non-blocking back pressure - for JVM and JavaScript runtime environments
    - https://www.reactive-streams.org/
  - **Project Reactor**
    - fourth-generation reactive library, based on the Reactive Streams specification, for building non-blocking applications on the JVM
    - https://projectreactor.io/

# COMMON FEATURES AND CONCEPTS [*]

- **Composability** and **readability**
- Data as a flow manipulated with a rich vocabulary of **operators**
- Nothing happens until you subscribe
- **Backpressure** or the ability for the consumer to signal the producer that the rate of emission is too high

[*] From: https://projectreactor.io/docs/core/release/reference/#getting-started

# COMPOSABILITY & READABILITY

- *Composability* as the ability to orchestrate multiple asynchronous tasks, in which we use results from previous tasks to feed input to subsequent ones
- The ability to orchestrate tasks is tightly coupled to the readability and maintainability of code
  - as the layers of asynchronous processes increase in both number and complexity, being able to compose and read code becomes increasingly difficult
  - recall the “callback hell” with the callback model

# ASSEMBLY LINE ANALOGY

- We can think of data processed by a reactive application as moving through an *assembly line*
  - the raw material pours from a *source* (the original *Publisher*) and ends up as a finished product ready to be pushed to the *consumer* (or *Subscriber*)
  - the raw material can go through various transformations and other intermediary steps or be part of a larger assembly line that aggregates intermediate pieces together
  - If there is a glitch or clogging at one point (perhaps boxing the products takes a disproportionately long time), the afflicted workstation can signal upstream to limit the flow of raw material

# OPERATORS

- Operators are like *workstations* in an assembly analogy
  - each operator adds behavior to a Publisher and wraps the previous step's Publisher into a new instance
- The whole chain is thus linked, such that data originates from the first Publisher and moves down the chain, transformed by each link
  - eventually, a Subscriber finishes the process
- Many kind of operators
  - from simple transformation and filtering to complex orchestration and error handling

# NOTHING HAPPENS UNTIL SUBSCRIBING

- In a Publisher chain typically data does *not* start pumping into it by default
  - first, the abstract description of the asynchronous process is created (*configuration stage*)
  - then, by subscribing, a Publisher is bound to a Subscriber, which *triggers the flow of data in the whole chain*
    - this is achieved internally by a single request signal from the Subscriber that is propagated upstream, all the way back to the source Publisher

# BACKPRESSURE

- Can be described in the assembly line analogy as a feedback signal sent up the line when a workstation processes more slowly than an upstream workstation
  - a subscriber can work in *unbounded mode* and let the source push all the data at its fastest achievable rate
  - or, *it can use the request mechanism to signal the source that it is ready to process at most n elements (=> backpressure)*
- This transforms the push model into a **push-pull hybrid**
  - the downstream can pull n elements from upstream if they are readily available
  - but if the elements are not ready, they get pushed by the upstream whenever they are produced
- More later (about ReactiveX)

# HOT VS COLD

- The Rx family of reactive libraries distinguishes two broad categories of reactive streams: **cold** and **hot**
- This distinction mainly has to do with how the reactive stream reacts to subscribers:
  - a *cold* stream starts a new stream for each Subscriber, including at the source of data
    - every subscriber typically gets all the elements of the streams independently from the moment they subscribe
  - a *hot* stream does not start from scratch for each Subscriber
    - rather, late subscribers receive signals emitted *after* they subscribed
    - some hot reactive streams can cache or replay the history of emissions totally or partially.
    - from a general perspective, a hot sequence can even emit when no subscriber is listening
      - an exception to “nothing happens before you subscribe”

# AN EXAMPLE: REACTIVE EXTENSIONS

- Rx has been originally introduced as a library on .NET
  - inspired to research work on functional reactive programming (FRP)
- Nowadays "reactive extension" are being developed for almost every programming language
  - http://reactivex.io/
    - *"The Observer pattern done right"*
    - "combination of the best ideas from the Observer pattern, the Iterator pattern, and functional programming"
  - Common ref for several lang: RxJS, RxJava, RxScala, RxPython..
  - Adopted by several web/mobile framework
    - both frontend and backend (e.g. Angular)
- Main vision explained in the paper:
  *Erik Meijer. 2012. Your mouse is a database. Commun. ACM 55, 5 (May 2012), 66-73.*

# CORE PROPERTIES

- Three core properties:
  - **Asynchronous and event-based**
    - Rx's mission statement is to simplify those programming models.
    - a key aspect for developing reactive user interfaces, reactive web app, cloud app - where asynchrony is quintessential.
  - **Composition**
    - simplifying in particular the composition of asynchronous computations
  - **Observable collections**
    - leveraging the active knowledge of programming model like LINQ, by looking at asynchronous computations as observable data sources
    - A mouse *"becomes a database of mouse moves and clicks"*: such asynchronous data sources are composed using various combinators in the LINQ sense, allowing things like filters, projections, joins, time-based operations, etc.

# RX PRINCIPLES

- Using Rx, we can
  1. represent multiple *asynchronous data streams* (**observable** streams) coming from diverse sources
     - e.g., stock quote, tweets, computer events, web service requests, etc.
  2. *subscribe* to the event stream using the **IObserver<T>** interface (on .NET)
     - the IObservable<T> interface notifies the subscribed IObserver<T> interface whenever an event occurs.
- Because observable sequences are data streams, we can query them using standard LINQ query operators implemented by the Observable extension methods.
  - thus you can filter, project, aggregate, compose and perform time-based operations on multiple events easily by using these standard LINQ operators.
- In addition, there are a number of other reactive stream specific operators that allow powerful queries to be written.
  - cancellation, exceptions, and synchronization are also handled gracefully by using the extension methods provided by Rx
- Rx complements and interoperates smoothly with both synchronous data streams (IEnumerable<T>) and single-value asynchronous computations (Task<T>) as the following diagram shows

# IOBSERVABLE IN .NET

- Rx complements and interoperates smoothly with both synchronous data streams (IEnumerable<T>) and single-value asynchronous computations (Task<T>) as the following diagram shows:

| | **Single return value** | **Multiple return values** |
|---|---|---|
| **Pull/Synchronous/Interactive** | T | IEnumerable<T> |
| **Push/Asynchronous/Reactive** | Task<T> | **IObservable<T>** |

# OBSERVER/OBSERVABLE INTERFACE

```
interface IObservable<T> {
  IDisposable Subscribe(IObserver<T> observer);
}
```

```
interface IObserver<T> {
  void OnNext(T value);
  void OnError(Exception error);
  void OnCompleted();
}
```

```
interface IDisposable {
  void Dispose();
}
```

# SIMPLE EXAMPLES IN .NET

```
…
IObservable<int> source = Observable.Range(1, 10);
IDisposable subscription = source.Subscribe(
    x => Console.WriteLine("OnNext: {0}", x),
    ex => Console.WriteLine("OnError: {0}", ex.Message),
    () => Console.WriteLine("OnCompleted"));
Console.WriteLine("Press ENTER to unsubscribe...");
Console.ReadLine();
subscription.Dispose();
…
```

- When an observer subscribes to an observable sequence, the thread calling the Subscribe method can be different from the thread in which the sequence runs till completion.
- Therefore, the Subscribe call is *asynchronous* in that the caller *is not blocked until the observation of the sequence completes*.

# QUERYING OBSERVABLE SEQUENCES USING LINQ OPERATORS

- Combining streams

```
var source1 = Observable.Range(1, 3);
var source2 = Observable.Range(1, 3);
source1.Concat(source2)
       .Subscribe(Console.WriteLine);
```

the resultant sequence is 1,2,3,1,2,3. This is because when you use the Concat operator, the 2nd sequence (source2) will not be active until after the 1st sequence (source1) has finished pushing all its values. It is only after source1 has completed, then source2 will start to push values to the resultant sequence. The subscriber will then get all the values from the resultant sequence.

- Merge streams

```
var source1 = Observable.Range(1, 3);
var source2 = Observable.Range(1, 3);
source1.Merge(source2)
       .Subscribe(Console.WriteLine);
```

in this case the resultant sequence will get 1,1,2,2,3,3. This is because the two sequences are active at the same time and values are pushed out as they occur in the sources. The resultant sequence only completes when the last source sequence has finished pushing values.

- Projection

```
var seqNum = Observable.Range(1, 5);
var seqString =
   from n in seqNum
    select new string('*', (int)n);
seqString.Subscribe(
  str => { Console.WriteLine(str); });
Console.ReadKey();
```

mapping a stream of num into a stream of strings

# QUERYING OBSERVABLE SEQUENCES USING LINQ OPERATORS (.NET)

- Other important and useful operators
  - filtering, time oriented operations, handling exceptions…
- In general:
  - **transform** data
    - using methods like aggregation and projection.
  - **compose** data
    - using methods like zip and selectMany.
  - **query** data
    - using methods like where and any.

# REFERENCES


- LEE, E. A. AND MESSERSCHMITT, D. G. 1987. Synchronous Data Flow. In Proceedings of the IEEE. Vol. 75. 1235–1245.
- ELLIOTT, C. AND HUDAK, P. 1997. Functional reactive animation. In Proceedings of the second ACM SIGPLAN International conference on Functional programming. ICFP ’97. ACM, New York, NY, USA, 263– 273.
- G. H. COOPER, S. KRISHNAMURTHI, S. 2006. Embedding dynamic dataflow in a call-by-value language. In ESOP’06: Proceedings of the 15th European conference on Programming Languages and Systems. Springer-Verlag, Berlin, Heidelberg, 294–308.
- EDWARDS, J. 2009. Coherent reaction. In OOPSLA ’09: Proceeding of the 24th ACM SIGPLAN conference companion on Object oriented programming systems languages and applications. ACM, New York, NY, USA, 925–932.
- Erik Meijer. 2012. Your mouse is a database. Commun. ACM 55, 5 (May 2012), 66-73.
- E. Bainomugisha et al. A Survey on Reactive Programming.ACM Computing Surveys (CSUR). Volume 45 Issue 4, August 2013
- G. Salvaneschi et al. Towards Distributed Reactive Programming. OOPSLA 2014
- R. Mogk et al. Fault-tolerant Distributed Reactive Programming. ECOOP 2018: 1:1-1:26